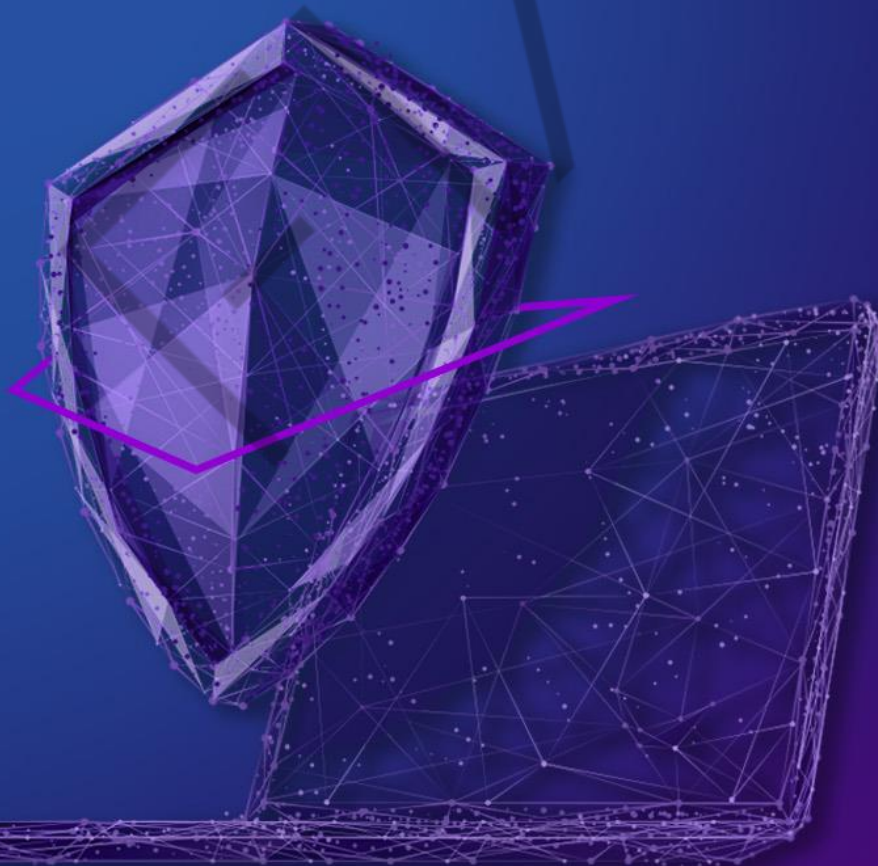


PYTHON O CÓDIGO DA
RESISTÊNCIA CONTRA O IMPÉRIO

INTRODUÇÃO A LINGUAGEM PYTHON



4

LISTA DE FIGURAS

Figura 1 – Modelo para representação de uma linguagem compilada.....	8
Figura 2 – Modelo para representação de uma linguagem interpretada.....	9
Figura 3 – Sequência de execução de código-fonte em Python.....	9
Figura 4 – Programa que realiza uma soma escrito em Python.....	12
Figura 5 – Programa que realiza uma soma escrito em C.....	12
Figura 6 – Página de downloads do site oficial do Python.....	13
Figura 7 – Página de arquivos para downloads do site oficial do Python.....	14
Figura 8 – Instalação do Python.....	14
Figura 9 – Versões do VS Code.....	16
Figura 10 – Adicionando extensões.....	16
Figura 11 – Adicionando extensões – Janela 2.....	17
Figura 12 – Adicionando uma pasta de trabalho.....	18
Figura 13 – Adicionando uma pasta de trabalho – Janela 2.....	18
Figura 14 – Adicionar arquivo.....	19
Figura 15 – Criação de arquivo.....	19
Figura 16 – Executando o código.....	20
Figura 17 – Primeiro projeto.....	20
Figura 18 – Instalação do módulo matplotlib.....	21
Figura 19 – Instalação do virtualenv.....	23
Figura 20 – Confirmação da instalação do virtualenv.....	23
Figura 21 – Ativação do virtualenv.....	24
Figura 22 – Versão do Pip.....	24
Figura 23 – Instalação do módulo matplotlib dentro do ambiente virtual.....	25
Figura 24 – Código gráfico.py sendo executado.....	25

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Verificando a versão do Pip	24
---	----

EMANIP

SUMÁRIO

1 INTRODUÇÃO	5
2 CONHECENDO O PYTHON.....	6
2.1 Linguagem simples e segura.....	6
3 ARQUITETURA DA LINGUAGEM	7
4 PYTHON, QUAL É O SEU PARADIGMA?.....	10
5 PYTHON É ALTO NÍVEL	11
5.1 Linguagem de alto e baixo nível.....	11
6 INSTALANDO O AMBIENTE DE TRABALHO	13
7 CONFIGURANDO O AMBIENTE DE TRABALHO.....	20
REFERÊNCIAS.....	27

1 INTRODUÇÃO

Python é a linguagem mais utilizada em segurança da informação, graças a sua versatilidade e facilidade na programação e, por isso, é utilizada em muitas ferramentas de segurança cibernética. Por ter uma grande variedade de bibliotecas, simplifica a construção de *scripts* para automatização de rotinas e de tarefas que necessitam de grande demanda de intervenção manual.

Atualmente, existem cerca de 24 bilhões de dispositivos interconectados pela internet ao redor do mundo e, com o advento da Internet de todas as coisas (*IoT*), a estimativa é que esse número cresça para 29 bilhões em 2023 (CISCO). Dessa forma, cada vez mais cresce a demanda por profissionais de segurança, que por sua vez, necessitam estar atualizados com as novas tecnologias e com a evolução da área.

O objetivo deste curso é introduzir os conceitos de programação de computadores, utilizando a linguagem Python, especificamente para o uso na segurança da informação e preparar o estudante para entender como a programação se aplica dentro deste moderno contexto. Vejamos alguns exemplos do enfoque que desejamos obter dentro desta disciplina:

- **Automatizar a geração de ativos de uma rede:** normalmente, essa tarefa é realizada nas grandes empresas por meio de alguma planilha em que um profissional será o responsável por documentar todos os ativos.
- **Alterar a configuração dos ativos de uma rede:** imagine um profissional que deverá alterar uma faixa de IP, porta, DNS ou qualquer outra informação importante para um ativo de rede, alterações essas que podem ocorrer frequentemente de acordo com as normas de segurança. Agora imagine um cenário pior, em que esses ativos estão geograficamente distantes, até mesmo em países diferentes. A solução será enviar um profissional para todas as filiais? Contratar em cada filial um profissional com essas habilidades? Ou permitir que um *script* possa ser executado remotamente, e que seja responsável por alterar as configurações desejadas?
- Realizar geração e leitura de *logs* locais com o objetivo de gerenciar e obter informações importantes sobre o comportamento dos equipamentos e

principalmente dos usuários, por questões de desempenho ou até mesmo por segurança.

- Desenvolver programas capazes de testar portas e endereçamentos lógicos, de maneira automática e frequente, dentro de uma rede, a fim de identificar possíveis vulnerabilidades.
- Enviar e/ou receber informações para equipamentos de IoT por meio de uma porta serial.
- Direcionar *patches* de atualizações e/ou correções de acordo com o sistema operacional que está sendo executado na máquina cliente, sem depender da interatividade dele.
- Criptografia.
- API.
- Construção de bots (como nosso chatbot no Telegram, por exemplo)!

2 CONHECENDO O PYTHON

2.1 Linguagem simples e segura

A linguagem Python teve sua primeira versão oficial em fevereiro de 1991, tendo como seu criador, o engenheiro de *software* Guido van Rossum, no instituto Centrum Wiskunde & Informatica (CWI) em Amsterdã, Holanda. A título de curiosidade, o autor deu esse nome devido a uma série de comédia britânica denominada *Monty Python's Flying Circus*.

Segundo o próprio Van Rossum (2015, tradução nossa):

Python é fácil de aprender e usar, e funciona de uma forma muito interativa. Os programadores de computador podem obter novos resultados muito rapidamente. Também é uma ótima ferramenta para cientistas que trabalham interativamente com dados. Além disso, é muito importante que, desde o início, Python esteja disponível de acordo com a filosofia do código aberto, mesmo antes de o termo "código aberto" ser inventado. Os usuários de Python se sentem parte de uma comunidade e são muito ativos em fazer melhorias.

Atualmente existem três linguagens de *scripting* populares, são elas: PHP, VBScript e Python. As duas primeiras são muito utilizadas na criação de aplicações *web*, porém não costumam ser utilizadas fora desse contexto.

Python, por ter uma sintaxe simples e clara, não somente é usada para *websites*, mas também em muitas outras aplicações. Grandes instituições como NASA, Google, Microsoft, Disney, Yahoo e Nokia utilizam Python em seus projetos.

Abaixo são relacionados alguns recursos interessantes que a linguagem oferece:

- Programação orientada a objetos;
- Tratamento de erros com o uso de *exceptions*;
- Limpeza de memória automática, similar ao *Garbage Collection* da linguagem Java;
- O uso de módulos para facilitar o reuso de código;
- Facilidade no uso de estruturas de dados complexas;
- Execução simplificada de aplicações em multiplataformas.

3 ARQUITETURA DA LINGUAGEM

Em uma linguagem de programação tradicional, assim como C ou C++, para que o código-fonte se torne um programa executável no sistema operacional é necessário um processo chamado compilação. Na figura abaixo é descrito o processo de compilação de forma sistematizada:

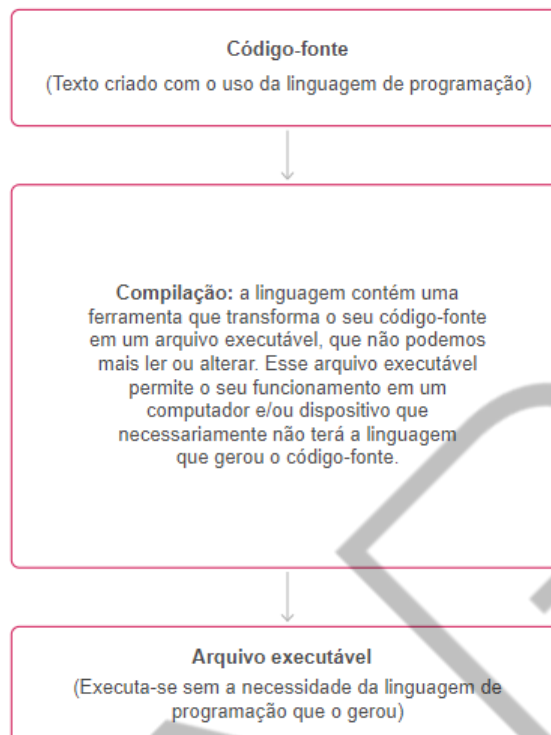


Figura 1 – Modelo para representação de uma linguagem compilada
Fonte: Elaborado pelos autores (2017)

Para exemplificar, imagine que você escreva um arquivo com comandos em C e gere um código-fonte “aplicacao.c”. Quando conclui o seu código, precisa executá-lo em outro computador, que não necessariamente terá a linguagem C instalada, então, você compila, ou seja, gera um arquivo executável no seu computador, normalmente com extensão “.exe”, o qual ficaria com o nome “aplicacao.exe”. Esse arquivo executável não depende mais da existência da linguagem C na máquina em que será executado.

Vejamos agora outra arquitetura utilizada pelas linguagens de programação. Um exemplo que utiliza essa arquitetura é a linguagem *Basic*, em que o código-fonte agora não será convertido, apenas será interpretado por alguma aplicação que a própria linguagem possui, em outros termos, caso queira executar sua aplicação, deverá instalar o responsável da linguagem que interpretará o seu código-fonte. Não será gerada outra forma de arquivo. É como ocorre com o HTML (que não é exatamente uma linguagem de programação, porém isso não é assunto para o momento) que depende de um *browser* para que seja executado. Sem um *browser*, um HTML é apenas um aglomerado de códigos.

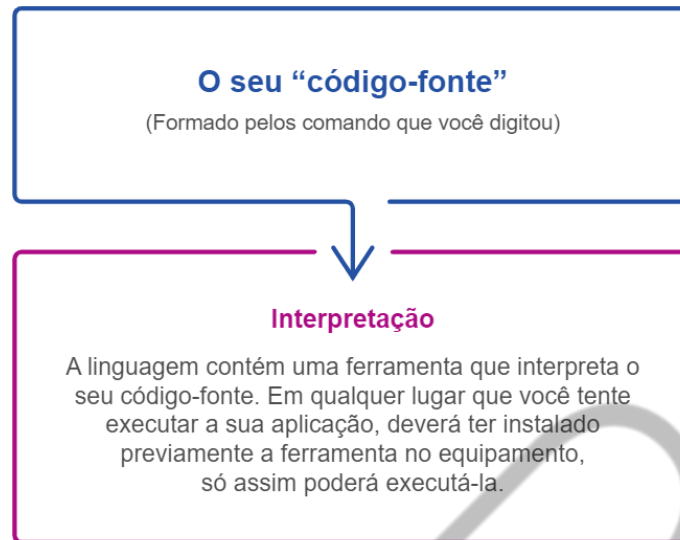


Figura 2 – Modelo para representação de uma linguagem interpretada
Fonte: Elaborado pelos autores (2017)

Bem, agora o que ocorre com Python? Python trabalha dentro da arquitetura interpretada, ou até mesmo com as duas arquiteturas juntas (compilada e interpretada), formando uma arquitetura híbrida, o que gera algumas discussões na comunidade: Python faz uso da arquitetura interpretada, compilada e/ou híbrida?

Em Python, após a criação de um novo código-fonte, isto é, após gerar um arquivo com extensão “.py”, o programador solicita ao interpretador a sua execução gerando um arquivo chamado *bytecode*, um arquivo com extensão “.pyc” ou um arquivo chamado *bytecode* otimizado, extensão “.pyo”.

Esse arquivo compilado é interpretado por uma ferramenta chamada PVM (*Python Virtual Machine*) que deverá estar instalada na máquina. Portanto, pode-se considerar que a arquitetura da linguagem seja híbrida, pois existe tanto o processo de compilação, quanto o de interpretação.

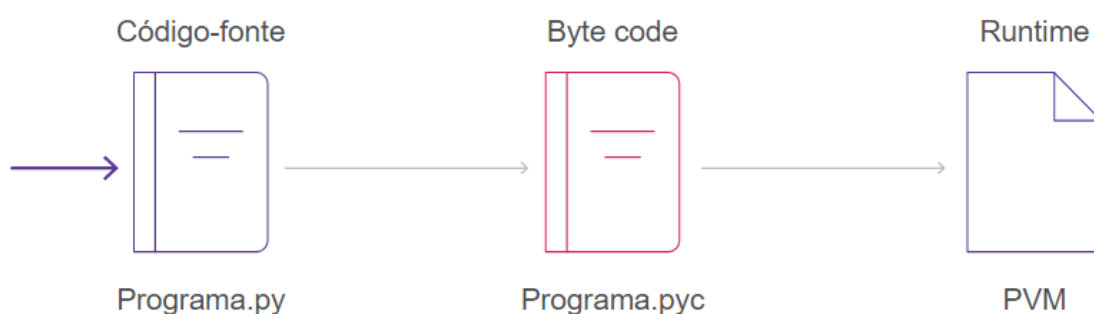


Figura 3 – Sequência de execução de código-fonte em Python
Fonte: Elaborado pelo autor (2020)

Outro ponto interessante é que, antes de compilar um código-fonte, a linguagem irá verificar se já existe uma versão do arquivo “.pyc” ou “.pyo” já compilada anteriormente, deixando o processamento do código ainda mais eficiente.

4 PYTHON, QUAL É O SEU PARADIGMA?

Um paradigma de programação é um conjunto de características que definem a metodologia, o estilo de programação de uma determinada linguagem de programação. Em outras palavras, que definem a forma como os problemas a serem resolvidos serão solucionados. Abaixo descreveremos alguns dos principais paradigmas de programação:

- **Paradigma imperativo ou estruturado:** como o próprio nome já diz, tem como princípio a estrutura rígida, muito ligado aos verbos imperativos: “faça isso, se não, faça aquilo”. Dessa forma, esse paradigma se torna uma forma simples de programar, muito próxima da ideia humana de executar suas ações. Exemplo: linguagens C, Pascal, Python, entre outras;
- **Paradigma orientado a objetos:** provavelmente o paradigma mais conhecido e utilizado, tem como premissa a solução dos problemas baseados nas características dos objetos reais trazidos para um modelo que simplifica a construção e interpretação dos códigos. Exemplo: linguagens PHP, Java, Python, entre outras;
- **Paradigma funcional:** baseado nas funções matemáticas como modelo para solução de problemas. Exemplo: linguagens Haskell, LISP, entre outras;
- **Paradigma lógico:** bastante utilizado atualmente em aplicações de inteligência artificial, utiliza-se de proposições e inferências em seu modelo lógico. Exemplo: linguagens Prolog, entre outras.

Apesar das diferenças existentes em cada paradigma, não se pode dizer que existe um melhor que o outro, mas sim, que existem vantagens e desvantagens em seus usos, principalmente em como cada paradigma se encaixa na solução de determinado problema.

Por exemplo, uma linguagem baseada em paradigma funcional pode se adequar melhor na solução de um problema estatístico do que uma linguagem orientada a objetos. Outro ponto é que algumas linguagens podem possuir mais de um paradigma, nesse caso são chamadas linguagens multiparadigmas.

Python é uma linguagem multiparadigma, pois suporta tanto o paradigma estruturado quanto o paradigma orientado a objeto. Nesta disciplina, iremos utilizar o paradigma estruturado, por isso, ele será abordado com um pouco mais de detalhes.

O paradigma estruturado faz uso basicamente de três pilares: sequência, decisão e iteração. Esses três pilares suportam e induzem a criação de rotinas simples que possuam poucas linhas de códigos ou, simplesmente, só o suficiente para que possam funcionar.

Imagine o seguinte: você precisa desenvolver diversas ferramentas para sua empresa: gerador de *backup*, criador de arquivos de *log*, leitor para arquivos de *log*, identificação de sistemas operacionais das máquinas clientes, testador de endereços IPs ou varredor de portas lógicas abertas.

Note que essas atividades tratam de problemas que necessitam de uma sequência lógica de comandos, baseadas em decisões, iterações e respostas. Portanto, é notório que o melhor paradigma a ser aplicado para essas atividades é o estruturado.

É dentro desse cenário que iremos explorar o conhecimento durante o curso. Serão abordados os assuntos referentes à lógica de programação e suas estruturas, sempre com o viés do paradigma estruturado, focado em resolver problemas ligados à segurança cibernética, utilizando a linguagem Python como ferramenta de trabalho.

5 PYTHON É ALTO NÍVEL

5.1 Linguagem de alto e baixo nível

Uma linguagem de programação de alto nível é aquela em que o código é mais legível à interpretação humana, em outras palavras, são linguagens que facilitam a programação por tratar com elementos mais próximos à linguagem humana.

Ao contrário, as linguagens de baixo nível são aquelas mais próximas do código interpretado pela máquina, em outras palavras, linguagens mais complexas para o entendimento humano, porém têm como vantagem serem interpretadas mais rapidamente pelo CPU.

A resposta para a pergunta é sim, Python é uma linguagem de alto nível e, para exemplificar, mostraremos um código pequeno feito em Python (alto nível) e como ele ficaria na linguagem C (baixo nível):

```
a = int(input("Digite um numero inteiro: "))
b = int(input("Digite um numero inteiro: "))

soma = a + b
print("\nO valor da soma = ", soma)
```

Figura 4 – Programa que realiza uma soma escrito em Python
Fonte: Elaborado pelo autor (2020)

```
#include <stdio.h>
#include <stdlib.h>
int main (void)
{
    int a, b, soma;

    printf("Digite um numero inteiro: ");
    scanf("%d", &a);

    printf("Digite um numero inteiro: ");
    scanf("%d", &b);

    soma = a + b;

    printf("O valor da soma = %d\n", soma);

    return(0);
}
```

Figura 5 – Programa que realiza uma soma escrito em C
Fonte: Elaborado pelo autor (2020)

Note a diferença que existe em um simples código que realiza uma soma de dois números inteiros. No código produzido em Python, existem apenas 4 linhas e é bem fácil de entender o que cada linha faz. Já o código feito em C é maior e um pouco mais difícil.

É por isso que a escolha da maioria das pessoas é por linguagens de alto nível como Python, dada a facilidade de entender sua sintaxe, interpretação e a redução de código para solucionar problemas.

Vale ressaltar que linguagens de baixo nível são muito úteis e até indispensáveis quando há poucos recursos computacionais, por exemplo, *softwares* rodando em dispositivos embarcados em sistemas eletrônicos de veículos, eletrodomésticos, sistemas de segurança, relógios digitais, entre outros. Nesses casos, o *hardware* que processa o código pode não ser suficiente para executar um programa feito em linguagem de alto nível.

Para o transcurso desta disciplina, faz-se necessário o uso de uma linguagem de alto nível para que seja possível desenvolver de uma forma simples e didática conceitos importantes e deixando a complexidade somente do lado do problema a ser resolvido e não nas ferramentas que usaremos para resolvê-lo.

6 INSTALANDO O AMBIENTE DE TRABALHO

Para iniciar qualquer desenvolvimento de *software* é necessária a instalação do ambiente de trabalho. Para esta disciplina utilizaremos a última versão do Python 3 e como editor de código, a IDE Visual Studio Code.

1º - Para baixar o Python 3, acesse o site oficial pela seguinte url:

[<https://www.python.org/downloads/>](https://www.python.org/downloads/)

Na página de *downloads*, busque por “*Looking for a specific release?*”, conforme figura abaixo:

Release version	Release date	Click for more
Python 3.9.0	Oct. 5, 2020	Download Release Notes
Python 3.8.6	Sept. 24, 2020	Download Release Notes
Python 3.5.10	Sept. 5, 2020	Download Release Notes
Python 3.7.9	Aug. 17, 2020	Download Release Notes
Python 3.6.12	Aug. 17, 2020	Download Release Notes
Python 3.8.5	July 20, 2020	Download Release Notes
Python 3.8.4	July 13, 2020	Download Release Notes
Python 3.7.8	June 27, 2020	Download Release Notes

[View older releases](#)

Figura 6 – Página de downloads do site oficial do Python
Fonte: <https://www.python.org/> (2020)

Dentro da página, vá até a seção “Files” e faça o *download* de acordo com o seu sistema operacional, conforme a imagem abaixo:

Version	Operating System	Description	MD5 Sum	File Size	PGP
Gzipped source tarball	Source release		e19e75ec81dd04de27797bf3f9d918fd	26724009	SIG
XZ compressed source tarball	Source release		6ebfe157fe88d9eabfbaf3fa92129f6	18866140	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	16ca86fa3467e75bade26b8a9703c27f	31132316	SIG
Windows help file	Windows		9ea6fc676f0fa3b95af3c5b3400120d6	8757017	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	60d0d94337ef657c2cca1d3d9a6dd94b	8387074	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	b61a33dc28f13b561452f3089c87eb63	28158664	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	733df85afb160482c5636ca09b89c4c8	1364352	SIG
Windows x86 embeddable zip file	Windows		d81fc534080e10bb4172ad7ae3da5247	7553872	SIG
Windows x86 executable installer	Windows		4a2812db8ab9f2e522c96c7728cfcbb	27066912	SIG
Windows x86 web-based installer	Windows		cdbfa799e6760c13d06d0c2374110aa3	1327384	SIG

Figura 7 – Página de arquivos para downloads do site oficial do Python
Fonte: <https://www.python.org/> (2020)

2º - Após a conclusão do *download*, execute o instalador e muita atenção à primeira tela da instalação:



Figura 8 – Instalação do Python
Fonte: <https://www.python.org/> (2020)

Na figura acima, deve ser selecionada a opção “*Add Python 3.x to PATH*”, porque irá adicionar as referências para a execução do Python, dentro das variáveis de ambiente do sistema operacional, por isso, é fundamental selecioná-la. Caso não o faça, poderá executar Python somente a partir da pasta em que você o instalou.

Em seguida, clique sobre a opção “*Install Now*”, caso queira aceitar todas as sugestões padrão, como ícones que estarão na sua área de trabalho, local de

instalação etc. Se, por outro lado, preferir alterar alguma dessas configurações, utilize a opção “*Customize Installation*”.

3º - Após a conclusão da instalação, abra o *prompt de comando*:

- Se estiver no Windows, utilize, <Windows> + <R>, digite cmd e pressione <enter>.
- No prompt de comando (CMD), digite “python” e, então, estará dentro do IDLE, que é o ambiente puro do Python para o desenvolvimento de um ambiente interativo, no qual já é possível programar.
- Digite **exit()** e pressione <enter> para sair do IDLE.
- Digite **exit** e pressione <enter> para sair do CMD.

Entretanto, não iremos programar dentro do IDLE, devido à sua falta de flexibilidade e praticidade em montar os códigos. Para isso, iremos utilizar o **Visual Studio Code (VS Code)**, que se enquadra na categoria IDE.

Os IDEs (*Integrated Development Environment* ou ambiente de desenvolvimento integrado) permitem programar em múltiplas linguagens e possuem inúmeros recursos que viabilizam a produtividade e a praticidade durante o desenvolvimento e os testes do seu programa.

4º - Para baixar o Visual Studio Code, acesse o site oficial pela seguinte url:

<<https://code.visualstudio.com>>

A página que será aberta apresentará a versão sugerida da IDE para o seu sistema operacional, seja ele Windows, Linux ou Mac OS, conforme podemos observar na imagem abaixo:

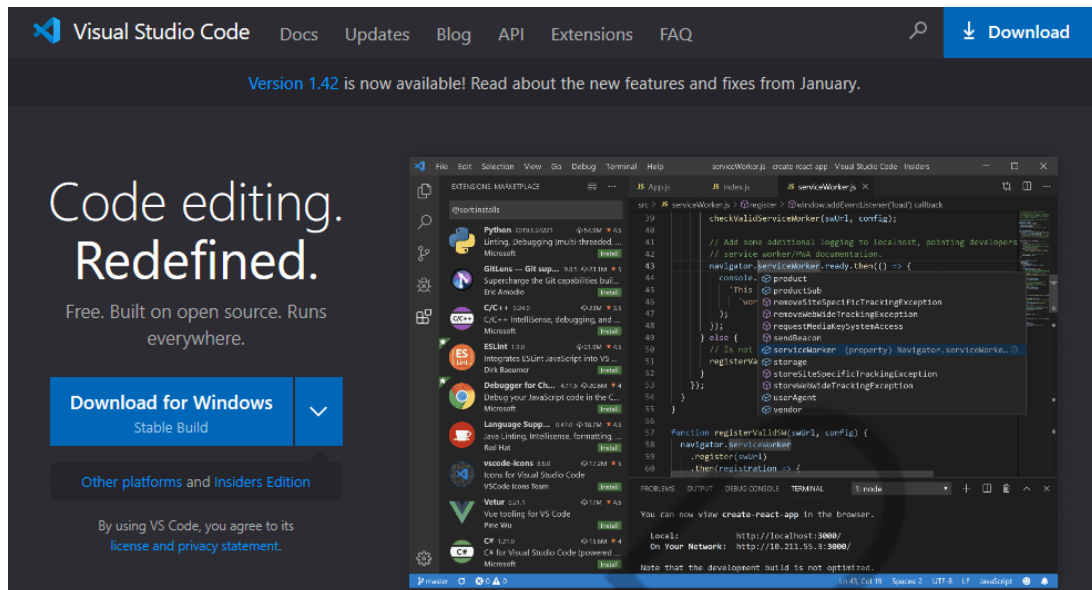


Figura 9 – Versões do VS Code.

Fonte: <https://code.visualstudio.com> (2020)

Após realizar o *download*, prossiga com a instalação padrão do VS Code. Não há qualquer observação relevante dentro do seu processo de instalação.

5º - Ao executar o VS Code pela primeira vez, deveremos adicionar a extensão referente à linguagem Python:

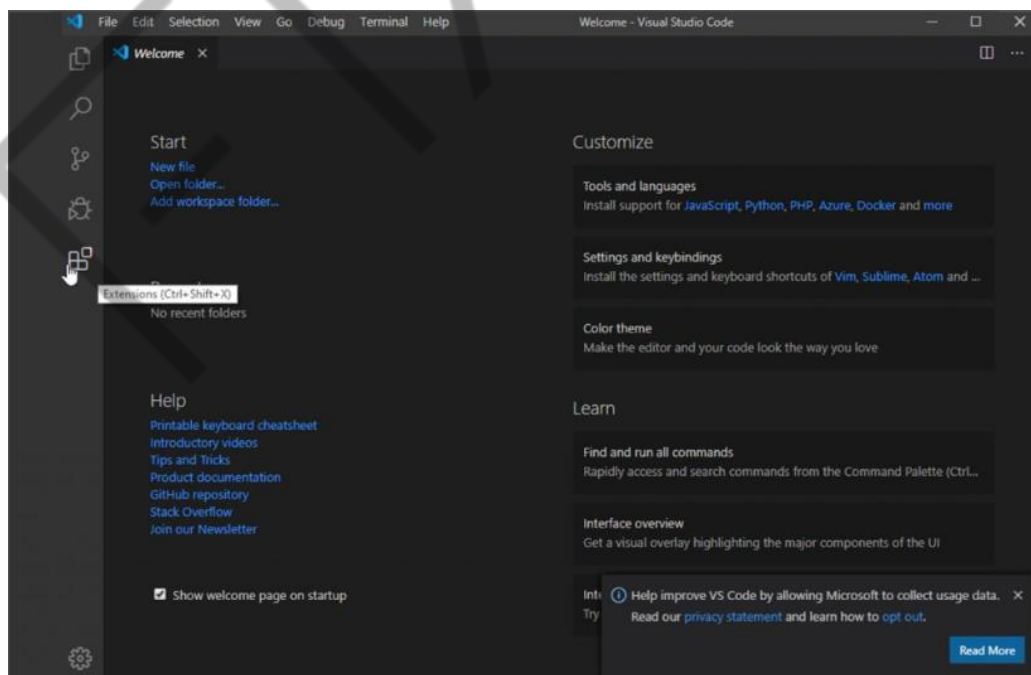


Figura 10 – Adicionando extensões

Fonte: VS Code (2020)

O último ícone do menu lateral esquerdo, refere-se à aba de extensão da IDE. Após abrir a aba, a primeira extensão sugerida é Python. Ao selecioná-la, podemos acionar o botão de instalação para instalar a extensão.

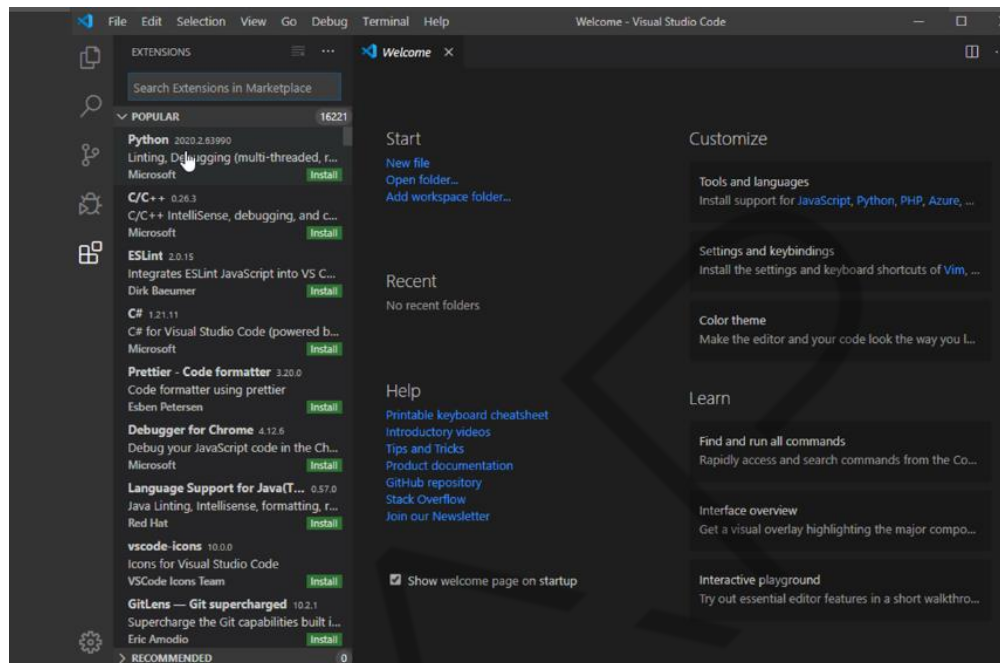


Figura 11 – Adicionando extensões – Janela 2
Fonte: VS Code (2020)

Finalizada a instalação, a IDE irá notificar que algumas configurações do ambiente estão pendentes, como o endereçamento do interpretador do Python ou algum pacote que necessita de instalação.

A própria notificação já apresenta opções de configurar automaticamente ou instalar os módulos, nesse caso podemos utilizar o processo automatizado sugerido pela própria IDE.

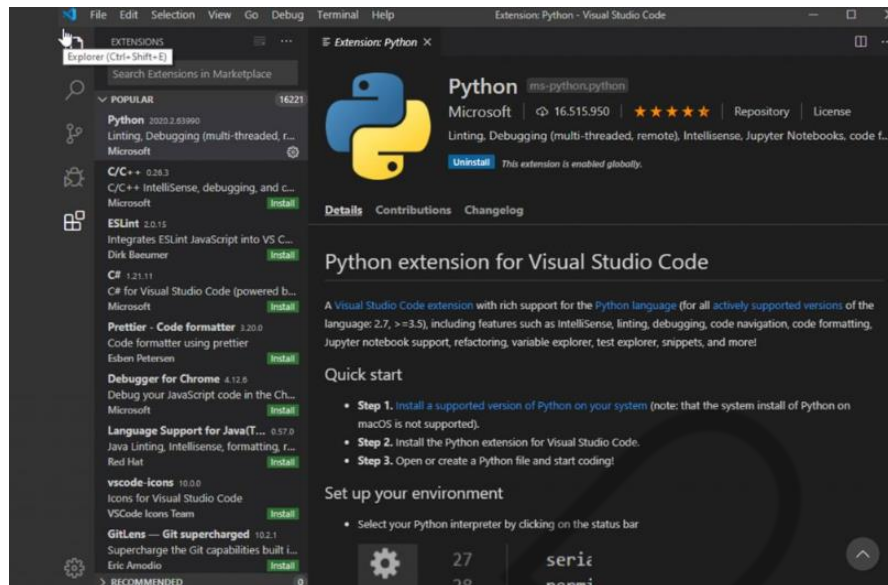


Figura 12 – Adicionando uma pasta de trabalho
Fonte: VS Code (2020)

Na figura acima, podemos selecionar a opção de explorar no menu lateral. Essa é a área do VS Code responsável por apresentar nossa estrutura de pastas e arquivos. No primeiro uso da IDE, ela ainda não possuirá uma pasta de trabalho adicionada. Para adicioná-la, devemos acionar o botão de “*Open Folder*” e selecionar uma pasta, como mostra a figura abaixo.

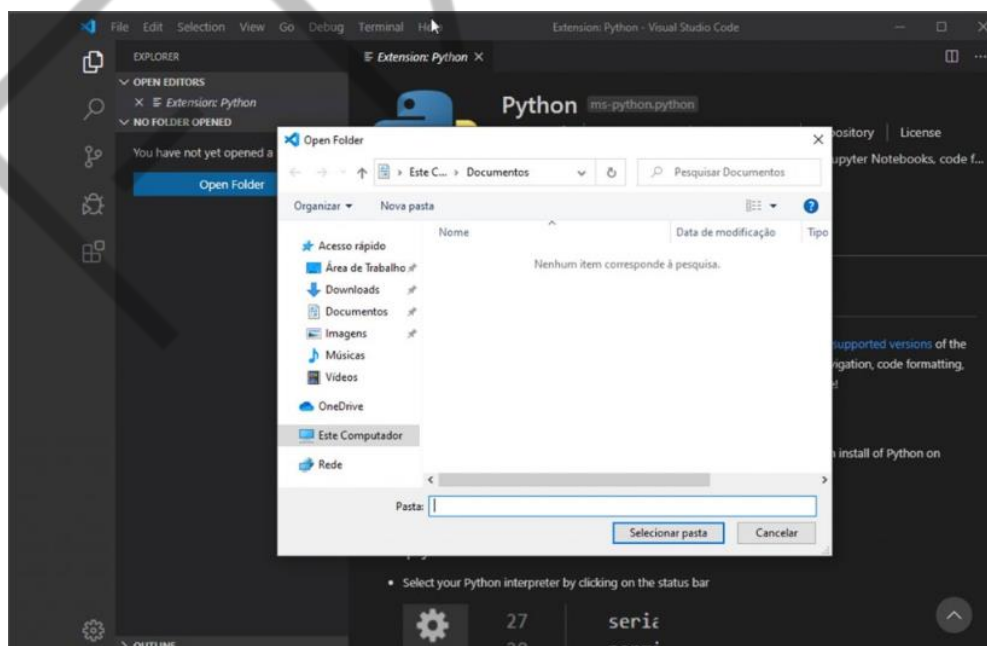


Figura 13 – Adicionando uma pasta de trabalho – Janela 2
Fonte: VS Code (2020)

Após selecionar uma pasta de trabalho, ela estará disponível por meio da aba lateral da IDE. A partir dela, podemos gerir a estrutura de pastas do nosso projeto, adicionando/removendo novos arquivos e subpastas. Para adicionar um arquivo ou pasta, podemos utilizar os ícones de adição rápida disponíveis na área de trabalho do projeto, como mostra a figura abaixo, ou clicando com o botão direito do mouse e selecionando a opção desejada.

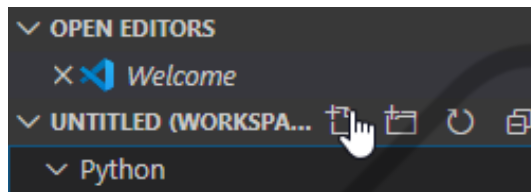


Figura 14 – Adicionar arquivo
Fonte: VS Code (2020)

Na criação do arquivo, precisamos adicionar a extensão da linguagem após o nome do arquivo. Dessa forma, se desejamos criar um arquivo de nome “hello_world”, devemos adicionar a extensão “.py” como apresenta a figura abaixo.

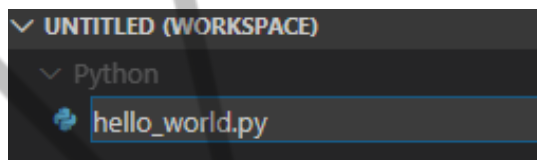


Figura 15 – Criação de arquivo
Fonte: VS Code (2020)

Depois de criar o primeiro arquivo, vamos criar nosso primeiro programa em Python. O primeiro programa que tipicamente criamos ao aprender uma nova linguagem é o “Hello World”. Para isso, vamos criar uma variável “msg” e passar o conteúdo “Hello World” para ela entre aspas. Dessa forma, Python irá entender que essa é uma variável de texto, ou *string*. Depois, em uma linha abaixo, vamos utilizar uma função de impressão “print” e colocar nossa variável entre parênteses. Para executar nosso código, basta clicar na seta verde no canto superior direito, conforme imagem abaixo:

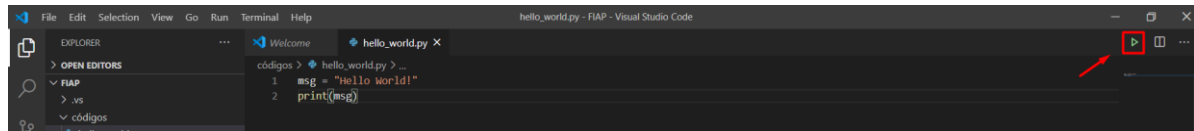


Figura 16 – Executando o código
Fonte: VS Code (2020)

Assim, a função irá apresentar no terminal da ferramenta o conteúdo na aba de console da IDE, como mostra a imagem abaixo:

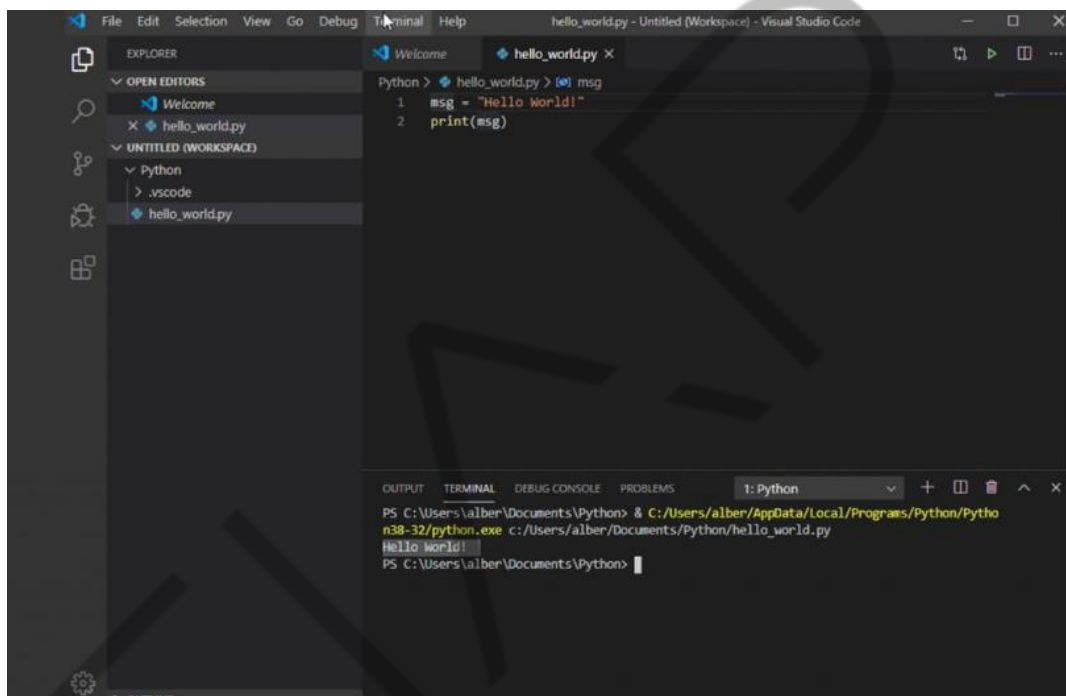


Figura 17 – Primeiro projeto
Fonte: VS Code (2020)

7 CONFIGURANDO O AMBIENTE DE TRABALHO

Durante o desenvolvimento de um programa, por diversas vezes será necessária a utilização de módulos externos desenvolvidos pela comunidade. Esses módulos são códigos já prontos que contêm uma série de funcionalidades que facilitam a programação.

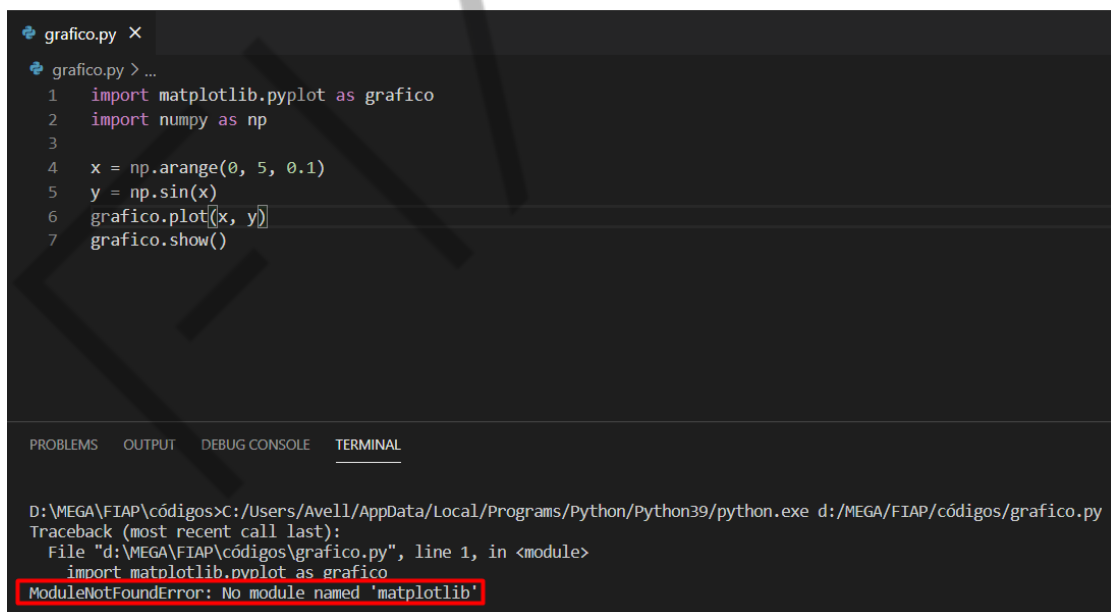
Imagine que você é um desenvolvedor em uma empresa e em determinado projeto que você trabalha é necessário que seu programa gere gráficos estatísticos a partir de dados armazenados no banco de dados.

Com as funções disponíveis na linguagem somente, não é possível. Para isso, você pode usar um módulo específico criado pela comunidade Python que lhe forneça essas funcionalidades. Nesse caso, por exemplo, pode-se usar uma biblioteca muito utilizada chamada `matplotlib`.

Para encontrar um módulo para um uso específico, é disponibilizado pela comunidade um repositório chamado *Python Package Index* (PyPI) onde é possível realizar pesquisas por mais de 270.000 projetos atualmente. Segue o link desse repositório:

<https://pypi.org/>

Depois que você encontrou o módulo que queria, você desenvolveu um pequeno código para verificar se ele funcionava corretamente, como na figura abaixo (não se preocupe em entender o código agora, esse assunto veremos mais adiante no curso). Note que após executar o código, foi apresentado no terminal, um erro informando que o módulo `matplotlib` não foi encontrado.



```
grafico.py X
grafico.py > ...
1 import matplotlib.pyplot as grafico
2 import numpy as np
3
4 x = np.arange(0, 5, 0.1)
5 y = np.sin(x)
6 grafico.plot(x, y)
7 grafico.show()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
D:\MEGA\FIAP\códigos>C:/Users/Avell/AppData/Local/Programs/Python/Python39/python.exe d:/MEGA/FIAP/códigos/grafico.py
Traceback (most recent call last):
  File "d:/MEGA/FIAP/códigos/grafico.py", line 1, in <module>
    import matplotlib.pyplot as grafico
ModuleNotFoundError: No module named 'matplotlib'
```

Figura 18 – Instalação do módulo `matplotlib`
Fonte: VS Code (2020)

Isso aconteceu porque é necessário instalar o módulo na máquina em que o código está sendo executado. Para instalar esse módulo também é necessário um instalador ou gerenciador de pacotes.

O programa instalador preferido pela maioria dos desenvolvedores e que vamos utilizar durante o curso é o **“Pip”**. Pip já vem incluído junto à instalação a partir da versão do Python 3.4.

Porém, antes de instalar qualquer módulo, é necessário explicar algo muito importante que é uma boa prática de programação. Por exemplo, imagine que estamos desenvolvendo o projeto A com a biblioteca matplotlib em sua versão 2.0.1 e o projeto B com a mesma biblioteca, porém na versão 3.3.3. Como seria feito para gerenciar essas dependências e o sistema operacional saber qual versão correta executar quando estivesse em cada projeto?

Para resolver isso, existe uma maneira muito eficiente e organizada que é a criação de um ambiente virtual para cada projeto. Um ambiente virtual armazena isoladamente todas as dependências de que um projeto precisa em um diretório, fazendo com que nenhum pacote seja instalado diretamente no sistema operacional, mas sim dentro do ambiente virtual daquele projeto. Sendo assim, cada projeto pode possuir suas bibliotecas em versões específicas.

Em Python, existe uma ferramenta responsável por criar esse ambiente virtual é a chamada **“virtualenv”**. Essa ferramenta pode ser instalada e utilizada diretamente no sistema operacional ou no próprio VS Code. Utilizaremos no nosso exemplo a instalação no VS Code. O uso direto no sistema operacional é importante para o aprendizado e fica como exercício para o aluno.

Para instalar o virtualenv no VS Code, vá até o terminal e digite:

```
py -3 -m venv <nome_diretorio>
```

No nosso exemplo, o comando é: “py -3 -m venv .venv”, como na figura abaixo:

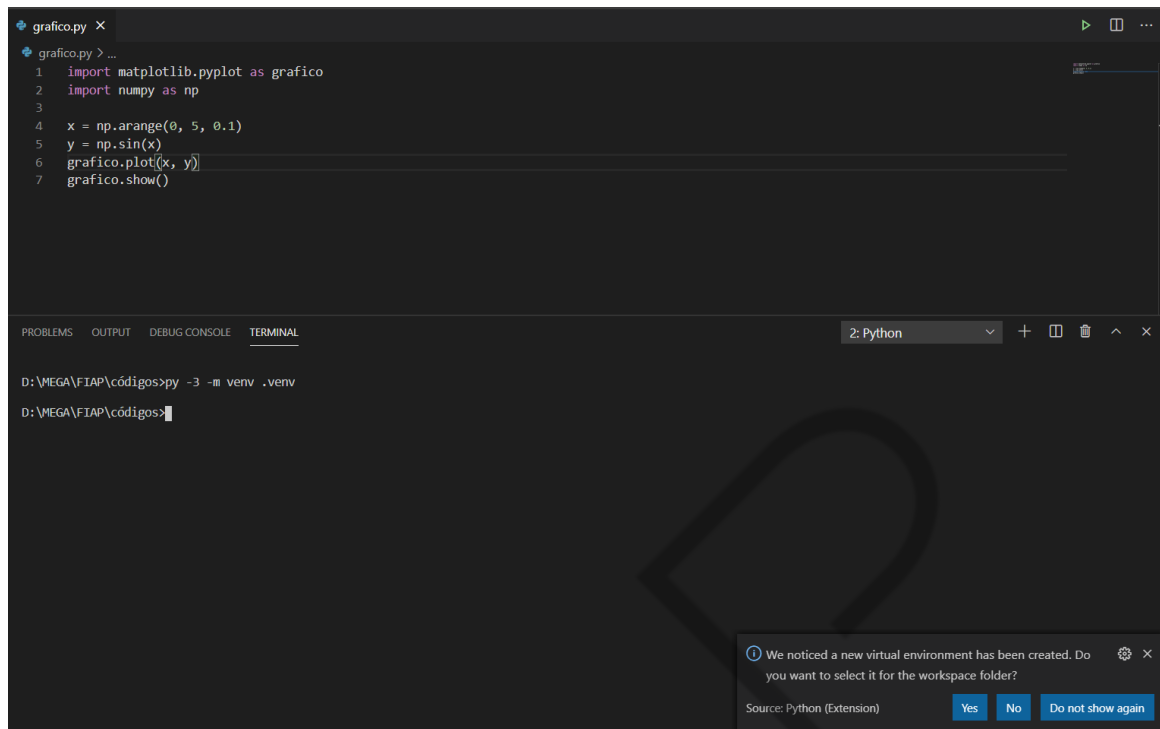


Figura 19 – Instalação do virtualenv
Fonte: VS Code (2020)

Note que, no canto inferior direito, o VS Code irá pedir uma confirmação da instalação do virtualenv. Clique em “Yes”.

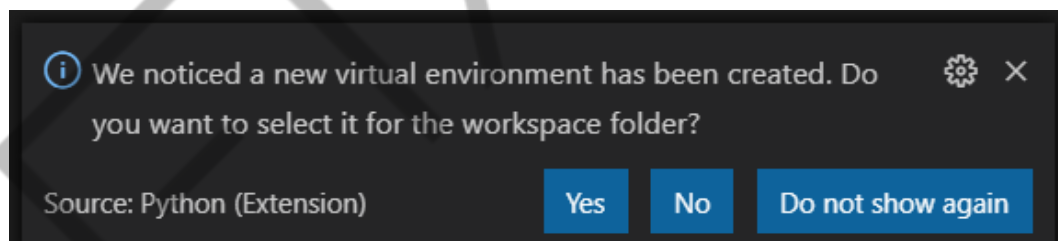
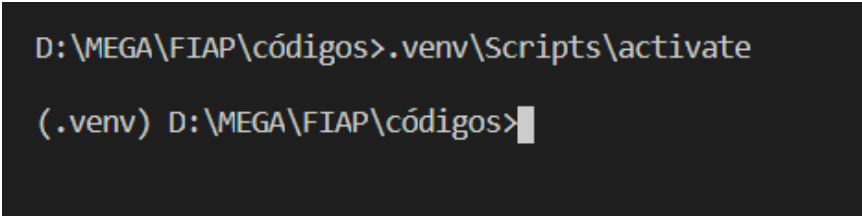


Figura 20 – Confirmação da instalação do virtualenv
Fonte: VS Code (2020)

Depois de instalado, o virtualenv precisa ser ativado para que possa utilizá-lo. Para isso, basta executar o script “activate” dentro do diretório criado. O comando abaixo é demonstrado na figura a seguir:

<nome_diretorio>\Scripts\activate



```
D:\MEGA\FIAP\códigos>.venv\Scripts\activate  
(.venv) D:\MEGA\FIAP\códigos>
```

Figura 21 – Ativação do virtualenv
Fonte: VS Code (2020)

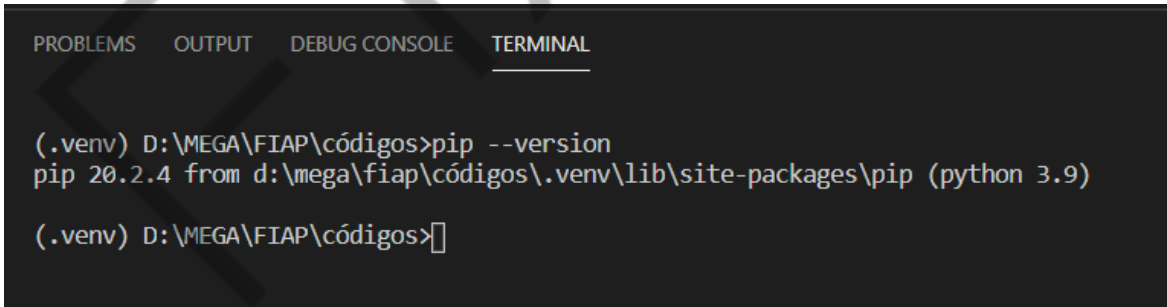
Note que a partir de agora o terminal apresenta o nome do diretório do virtualenv criado (.venv), antes do caminho do diretório principal, indicando que você está utilizando o ambiente virtual.

Pronto, com o ambiente virtual funcionando corretamente, podemos instalar os pacotes sem que tenhamos problemas de dependências no futuro. Retomando ao nosso exemplo anterior, já podemos instalar a biblioteca matplotlib utilizando o Pip dentro do nosso ambiente virtual.

Para verificar se o Pip está funcionando corretamente, digite:

```
pip --version
```

Comando de prompt 1 – Verificando a versão do Pip
Fonte: Elaborado pelo autor (2020)



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL  
(.venv) D:\MEGA\FIAP\códigos>pip --version  
pip 20.2.4 from d:\mega\fiap\códigos\.venv\lib\site-packages\pip (python 3.9)  
(.venv) D:\MEGA\FIAP\códigos>
```

Figura 22 – Versão do Pip
Fonte: VS Code (2020)

Estando o Pip funcionando corretamente, para instalar o módulo é simples:

```
pip install <nome_modulo>
```

No nosso exemplo, o comando fica assim: “pip install matplotlib”, como na figura abaixo:

```
(.venv) D:\MEGA\FIAP\códigos>pip install matplotlib
Collecting matplotlib
  Using cached matplotlib-3.3.3-cp39-cp39-win_amd64.whl (8.5 MB)
Requirement already satisfied: kiwisolver>=1.0.1 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: python-dateutil>=2.1 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (2.8.1)
Requirement already satisfied: pillow>=6.2.0 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (8.0.1)
Requirement already satisfied: numpy>=1.15 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (1.19.4)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (2.4.7)
Requirement already satisfied: cycler>=0.10 in d:\mega\fiap\códigos\.venv\lib\site-packages (from matplotlib) (0.10.0)
Requirement already satisfied: six>=1.5 in d:\mega\fiap\códigos\.venv\lib\site-packages (from python-dateutil>=2.1->matplotlib) (1.15.0)
Installing collected packages: matplotlib
Successfully installed matplotlib-3.3.3

(.venv) D:\MEGA\FIAP\códigos>
```

Figura 23 – Instalação do módulo matplotlib dentro do ambiente virtual
Fonte: VS Code (2020)

Pronto, o módulo já está instalado. Agora ao executar o código no VS Code, note que o código funcionou perfeitamente e o gráfico já pode ser visualizado:

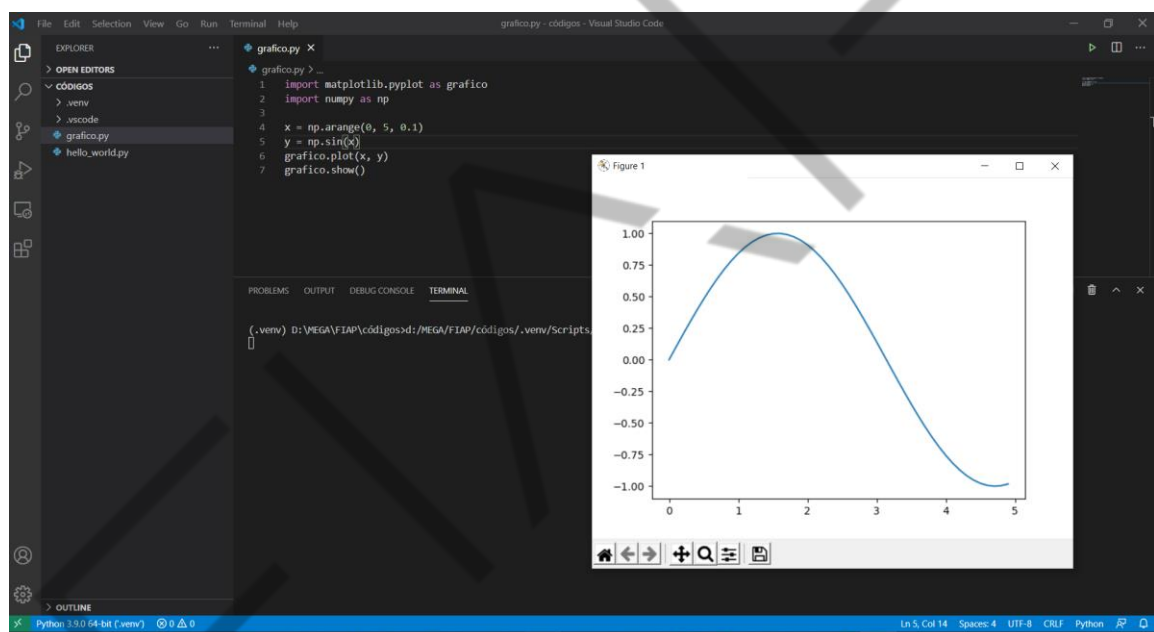


Figura 24 – Código gráfico.py sendo executado
Fonte: VS Code (2020)

Achou interessante? Agora imagine a quantidade de coisas diferentes que podemos criar ou até mesmo facilitar nosso trabalho cotidiano utilizando os módulos externos de Python.

Como exercício para o aluno, sugiro realizar buscas no repositório PyPI por módulos que ache interessante, instale em um novo ambiente virtual e tente testá-lo com trechos de código simples. Caso tenha dificuldade em criar o código, tente buscar códigos feitos pela comunidade postados nos diversos fóruns ou repositórios, como por exemplo o Github, segue a url:

```
<https://github.com/>
```

Por enquanto é só. Posteriormente, serão explicados os primeiros passos de lógica de programação. Reviva os principais conceitos desta etapa e incorpore-os no seu dia a dia. Até o próximo capítulo!

EMENDAS

REFERÊNCIAS

BORGES, L. E. **Python para Desenvolvedores**. São Paulo: Novatec, 2014.

CISCO. **Cisco Annual Internet Report (2018-2023) White Paper**. Mar 2020. Disponível em: <<https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>>. Acesso em: 4 mar. 2021.

PYTHON BRASIL. **Aprenda a programar**. Disponível em: <<https://wiki.python.org.br/AprendaProgramar>>. Acesso em: 4 mar. 2021.

PYTHON. **The Python Package Index**. Disponível em: <<https://pypi.org/>>. Acesso em: 4 mar. 2021.

SEITZ, J. **Black Hat Python**: Programação Python para hackers e pentesters. São Paulo: Novatec, 2015.

VAN ROSSUM, G. **25 year of Python at CWI**. CWI, 2015. Disponível em: <<https://www.cwi.nl/news/2015/25-years-of-python-at-cwi>>. Acesso em: 4 mar. 2021.

VISUAL STUDIO CODE. [s.d.]. Disponível em: <<https://code.visualstudio.com>>. Acesso em: 4 mar. 2021.